

Industrial Training
On
CivilInc: Design and Development of an AI-Enabled Civic Grievance
Redressal and Urban Infrastructure Management Platform for Smart
City Governance
Submitted By

Student Name: Yuvaraj S Yali

Section:

Reg No:

E-Mail Id: syaliyuvaraj@gmail.com

Roll No:

Mobile No: +91

Under the Guidance of:

Guide's Name: Smt. Kavitha Yogappanavar

Internal Guide's Name:

Designation: Commissioner, Shivamogga City Corporation
& Managing Director, Shivamogga Smart City Ltd.

Designation:

Name of the Organization: Shivamogga Smart City Limited

Manipal Institute of Technology



MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

School / Department name as required by your programme

Month Year

LETTER OF RECOMMENDATION

ಶಿವಮೊಗ್ಗ ಸ್ಮಾರ್ಟ್ ಸಿಟಿ ಲಿಮಿಟೆಡ್
ನೋಂದಾಯಣಾಂಕ: 18/ ಮಹಡಿ, ವಾಣಿ ಕಲ್ಲು ಬ್ಲಾಕ್,
ಎಸ್.ಎಸ್. ಮಾರ್ಕೆಟ್, ಶಿವಮೊಗ್ಗ - 577201



SHIVAMOGGA SMART CITY
LIMITED

Regd Office: 1st Floor, Corporation Building block,
SN Market, Nehru Road, Shivamogga - 577201

CIN: U74999KA2017PLC100268, GSTIN: 29AAYCS3808B1Z0, PAN: AAYCS3808B, TAN: BLRS63059D
Web: <http://shivamoggasmartcity.co.in/> E-mail: shivogasmartcity@gmail.com Phone No: 08182-279951

No: SSCL/ICT/2025-26/32

Date: 06-06-2025

To Whomsoever May Concern

This is to certify and recommend **Mr. Yuvaraj S Yali**, a student of Manipal Institute of Technology, for his remarkable initiative in independently conceptualizing, designing, and developing a civic-tech prototype titled "**Civilinc**".

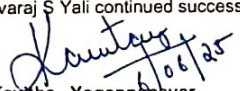
Civilinc is a self-developed prototype that showcases Mr. Yuvaraj's keen understanding of urban governance challenges, digital transformation, and citizen-centric service delivery. The platform has been designed to enhance the operational efficiency of municipal departments by streamlining complaint management, improving transparency, and enabling data-driven oversight through digital dashboards and smart workflows.

Having reviewed the scope and features of *Civilinc*, it is evident that the platform has the potential to deliver meaningful improvements if integrated into the existing systems of Shimoga City Corporation and Smart City company Ltd Shimoga. The following value propositions make *Civilinc* particularly noteworthy:

- **End-to-end digital grievance redressal system** for citizens and officials.
- **Cross-departmental coordination** through a centralized platform.
- **Real-time analytics and dashboards** for performance tracking and decision-making.
- **Transparency and accountability**, aligning with Smart City goals and governance reforms.

The fact that this prototype was **entirely built by Mr. Yuvaraj.S.Yali**, individually is a testament to his technical capability, design thinking, and commitment to civic innovation. We find his work both commendable and promising for practical pilot implementation and future development.

We strongly recommend *Civilinc* for consideration in civic tech innovation pipelines and wish Mr. Yuvaraj S Yali continued success in his academic and professional journey.


Smt.Kavitha . Yogappa
Commissioner – Shimoga City Corporation
Managing Director – Shimoga Smart City Limited

Letter No. SSCL/ICT/2025-26/32 dated 06-06-2025 issued by the Commissioner, Shivamogga City Corporation and Managing Director, Shivamogga Smart City Limited.

CONTENTS

S.No	Title	Page Number
1	Acknowledgement	(iii)
2	Details of the Organization	(iv)
3	List of Figures	(v)
4	List of Tables	(vi)
5	Chapter 1: Introduction and the Objectives of the Project	1-2
6	Chapter 2: Work Done during the Training Period	3-20
7	Chapter 3: Learnings and Impact of the ITR	21
8	Chapter 4: Concluding Remarks	22
9	References	23
10	Plagiarism / Similarity Report	24
11	Annexures	25

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Shivamogga Smart City Limited** and the **Shivamogga City Corporation** for giving me the opportunity to undertake this industrial training and to work on a problem of real public importance: how a city listens to, and acts upon, the grievances of its citizens. Working alongside a government organisation responsible for urban services gave me an understanding of civic administration that no classroom could have provided.

I am deeply grateful to **Smt. Kavitha Yogappanavar**, Commissioner, Shivamogga City Corporation and Managing Director, Shivamogga Smart City Limited, for her guidance and for reviewing the scope and features of the CivilInc prototype. Her recognition of the work, and her recommendation of the platform for consideration in civic-tech innovation pipelines, has been a great source of motivation for me.

I also thank the officers and staff of the ICT section of Shivamogga Smart City Limited and the various departments of the City Corporation who patiently explained how complaints are received, routed, escalated and closed today, and the practical difficulties they face in coordinating across departments. Their insights shaped the design of the platform.

I would like to thank my internal guide, **Name, Designation**, and the faculty of Manipal Institute of Technology for their support and for providing the academic framework within which this training was carried out.

Finally, I thank my family and friends for their constant encouragement throughout the internship and the development of this project.

YUVARAJ S YALI

DETAILS OF THE ORGANIZATION – SHIVAMOGGA SMART CITY LIMITED

Shivamogga Smart City Limited (SSCL) is a Special Purpose Vehicle (SPV) incorporated under the Companies Act, 2013 (CIN: U74999KA2017PLC100268) to plan, appraise, approve, fund, implement, manage and operate projects under the Government of India's **Smart Cities Mission** in Shivamogga, Karnataka. The company's registered office is located on the 1st Floor of the Corporation Building block, SN Market, Nehru Road, Shivamogga – 577201.

The Smart Cities Mission, launched by the Ministry of Housing and Urban Affairs, aims to promote cities that provide core infrastructure, a clean and sustainable environment and a decent quality of life to their citizens through the application of “smart solutions”. As the SPV for Shivamogga, SSCL executes area-based development and pan-city projects in domains such as roads and mobility, water supply and drainage, public spaces and parks, solid waste management, e-governance and Information & Communication Technology (ICT).

SSCL works in close coordination with the **Shivamogga City Corporation**, the urban local body responsible for delivering municipal services such as roads, street lighting, water supply, drainage, sanitation, building approvals and public health to the citizens of Shivamogga. The Commissioner of the City Corporation also serves as the Managing Director of SSCL, which allows smart city initiatives to be aligned directly with the day-to-day operational priorities of the municipal administration.

The **ICT division** of SSCL is responsible for the digital initiatives of the city, including integrated command-and-control capabilities, citizen service delivery, data-driven monitoring of civic services and digital transformation of municipal workflows. The industrial training described in this report was carried out in this context, with a focus on improving grievance redressal, cross-departmental coordination and performance monitoring through a digital platform.

LIST OF FIGURES

Figure Number	Description	Page Number
2.1	Layered System Architecture of the CivilInc Platform	4
2.2	Secure Sign-in Screen with Role-based Demo Accounts	6
2.3	Complaint Lifecycle and State Transitions in CivilInc	7
2.4	Citizen Complaint Registration Form with Location Details	8
2.5	Complaint Register with Search and Filters by Status, Priority and Category	8
2.6	Complaint Detail View with Status Update, Escalation and SLA Information	9
2.7	Commissioner Dashboard showing City-wide KPIs and High-Risk Wards	10
2.8	Engineer Dashboard with Assigned Complaints and At-Risk Projects	10
2.9	Coordinator Dashboard showing the Pending Triage Queue	11
2.10	Citizen Dashboard with Quick Actions and Complaint Tracking	11
2.11	Infrastructure Project Register with Progress, Delay Risk and Budget	12
2.12	Project Detail with Progress Update and AI Risk Assessment	13
2.13	GIS Map showing Geo-tagged Complaints colour-coded by Priority	14
2.14	Analytics Module with Category, Project Status and Budget Metrics	14
2.15	Department Overview with Complaints, Projects, SLA and Budget	15
2.16	Inter-departmental Discussion Forum	16
2.17	Data Engineering and Machine-Learning Pipeline of CivilInc	17
2.18	Summary of the Model Evaluation Report produced by the Training Pipeline	19
2.19	Live AI Triage of a New Complaint through the REST API	19
2.20	Commissioner AI Assistant answering a Backlog Query from Live Data	20

LIST OF TABLES

Table Number	Description	Page Number
2.1	Stakeholders and their requirements	3
2.2	Technology stack used in CivilInc	4
2.3	Core database tables	5
2.4	Summary of the role-permission matrix	5
2.5	Datasets produced by the ETL pipeline	16
2.6	AI systems implemented in CivilInc	17

CHAPTER 1

INTRODUCTION

1.1 Introduction

Indian cities are growing rapidly, and with this growth comes a steadily increasing volume of civic complaints: potholes, water supply disruptions, blocked drains, non-functional streetlights, uncleared garbage and unsafe buildings. In most urban local bodies these complaints are still received through a mix of phone calls, walk-in petitions, WhatsApp messages and isolated web portals. Each department maintains its own records, routing is manual, and there is little visibility into how long a complaint has been pending, whether service-level timelines are being met, or which wards are under the greatest stress.

During my industrial training with **Shivamogga Smart City Limited**, I studied how grievances and infrastructure projects are handled across departments of the City Corporation, identified the bottlenecks in the existing process, and independently conceptualised, designed and developed a civic-tech prototype titled “**Civillnc**”. Civillnc is a full-stack, role-based platform that brings citizens, field engineers, departmental coordinators and the Commissioner onto a single system for complaint management, project monitoring, GIS-based visualisation, analytics and AI-assisted decision making.

The platform was built using a modern technology stack: **Vue 3, TypeScript and Tailwind CSS** for the frontend; **FastAPI, SQLAlchemy and PostgreSQL** for the backend; and **scikit-learn, LightGBM and XGBoost** for seven machine-learning systems that automatically categorise and prioritise complaints, predict resolution times, detect complaint clusters, forecast project delays and budget overruns, and compute ward-level risk scores. The prototype is containerised with Docker and configured for continuous integration using GitHub Actions.

The prototype was reviewed by the Commissioner, Shivamogga City Corporation and Managing Director, Shivamogga Smart City Limited, who issued a letter of recommendation (No. SSCL/ICT/2025-26/32, dated 06-06-2025) noting that the platform “has the potential to deliver meaningful improvements if integrated into the existing systems of Shimoga City Corporation and Smart City company ltd Shimoga”, and recommending it for consideration in civic-tech innovation pipelines.

1.2 Problem Statement

Based on discussions with officials and a study of the existing grievance process, the following problems were identified:

- **Fragmented intake:** complaints arrive through many channels and are not consolidated in one system, which makes duplicates common and tracking difficult.
- **Manual routing and triage:** an official must read each complaint, decide its category, the responsible department and its urgency, which introduces delay and inconsistency.
- **No SLA visibility:** there is no automatic tracking of how long a complaint has been open against the expected resolution time, and escalation is ad hoc.
- **Weak cross-departmental coordination:** issues such as a water-pipe burst damaging a road involve multiple departments, but there is no shared workspace for them.
- **Limited oversight for leadership:** the Commissioner lacks a real-time, city-wide view of complaint backlogs, project progress, budget utilisation and ward-level risk.

- **Low transparency for citizens:** citizens cannot easily see the status of their complaint or the progress of public works in their ward.

1.3 Objectives of the Training Program

The key objectives of the industrial training were:

- To understand the organisational structure, service delivery processes and grievance-handling workflows of an urban local body and a Smart City SPV.
- To translate these real-world requirements into a software design with clearly defined user roles, permissions and workflows.
- To design and implement an **end-to-end digital grievance redressal system** covering complaint registration, triage, assignment, escalation, resolution and citizen feedback.
- To enable **cross-departmental coordination** through a centralised platform with shared dashboards, a discussion forum and real-time notifications.
- To build **real-time analytics, dashboards and GIS maps** for performance tracking and data-driven decision-making.
- To apply data engineering and machine learning to automate triage and to predict infrastructure risks such as project delays and budget overruns.
- To follow industry practices in security (JWT, RBAC, rate limiting, audit logging), testing, containerisation and CI/CD.

1.4 Training Methodology

The industrial training followed a structured, phased approach combining field study, design and hands-on implementation.

Phase 1 – Domain study and requirement gathering. The initial phase focused on understanding how the City Corporation and SSCL operate: the departments involved in civic services, the types of complaints received, the informal service-level expectations for each department, the escalation hierarchy and the reporting needs of senior officials. Interactions with officials helped identify the four primary user personas of the system – the **Commissioner**, the **Engineer**, the **Coordinator** and the **Citizen**.

Phase 2 – System design. In the second phase, the requirements were converted into a layered architecture, a relational database schema, a role-permission matrix, a complaint lifecycle state machine and a REST API contract. Technology choices were made with emphasis on open-source tools, scalability and ease of deployment for a government environment.

Phase 3 – Data engineering and AI. Since live municipal data could not be shared for a student prototype, a realistic dataset was engineered from public reference sources and published statistical distributions. ETL pipelines, data quality checks and seven machine-learning systems were developed and evaluated on this data.

Phase 4 – Implementation, integration and review. The backend, frontend and AI inference layer were implemented and integrated, role-specific dashboards were built, and the system was containerised and set up for CI/CD. The completed prototype was demonstrated and its scope and features were reviewed by the Commissioner, leading to the letter of recommendation.

CHAPTER 2

WORK DONE DURING THE TRAINING PERIOD

2.1 Study of the Existing Grievance Process and Requirements

The first part of the training focused on understanding the current way in which civic grievances move through the municipal system. A typical complaint – for example a broken streetlight or a burst water pipe – is reported by a citizen through a phone call, a visit to the ward office, a social-media message or a portal. It is then manually forwarded to the concerned department, where an engineer is eventually assigned to inspect and fix it. Status updates are rarely communicated back to the citizen, and the senior administration typically sees consolidated figures only in periodic review meetings.

From this study, the functional requirements of the platform were defined for each stakeholder:

Table 2.1: Stakeholders and their requirements

Stakeholder	Key requirements
Citizen	Register complaints with location and description; track status; receive notifications; rate the resolution; view public projects in the ward; take part in public discussions.
Coordinator	Review and validate incoming complaints; correct category and priority; assign to the right department or officer; monitor the department queue.
Engineer	View assigned complaints; update progress and resolution notes; manage infrastructure projects (progress, cost); see AI risk alerts for projects.
Commissioner	City-wide dashboard of complaints, SLA breaches, projects and budgets; department performance; ward-level risk; ability to escalate; natural-language assistant for quick insights.

Non-functional requirements included security of citizen data, role-based access control, auditability of every action, responsiveness on low-bandwidth connections, and the ability to deploy the system on standard cloud or on-premise infrastructure.

2.2 System Architecture

Civillnc follows a **layered, service-oriented architecture**. The presentation layer is a single-page application built with Vue 3. It communicates with a FastAPI backend over REST (JSON) for regular operations and over WebSockets for real-time events. The backend is organised into API routers, a service layer containing business logic (SLA computation, workflow transitions, feedback, auditing) and a repository layer that encapsulates all database access through SQLAlchemy 2.0 with asynchronous drivers.

Persistent data is stored in **PostgreSQL 16**, while **Redis 7** is provisioned for caching and publish-subscribe messaging. An independent AI/ML layer contains the training pipelines and serialised models, which are loaded lazily by an inference service inside the backend. All components are containerised using Docker Compose, with an Nginx configuration for production and a GitHub Actions workflow for automated testing and deployment.

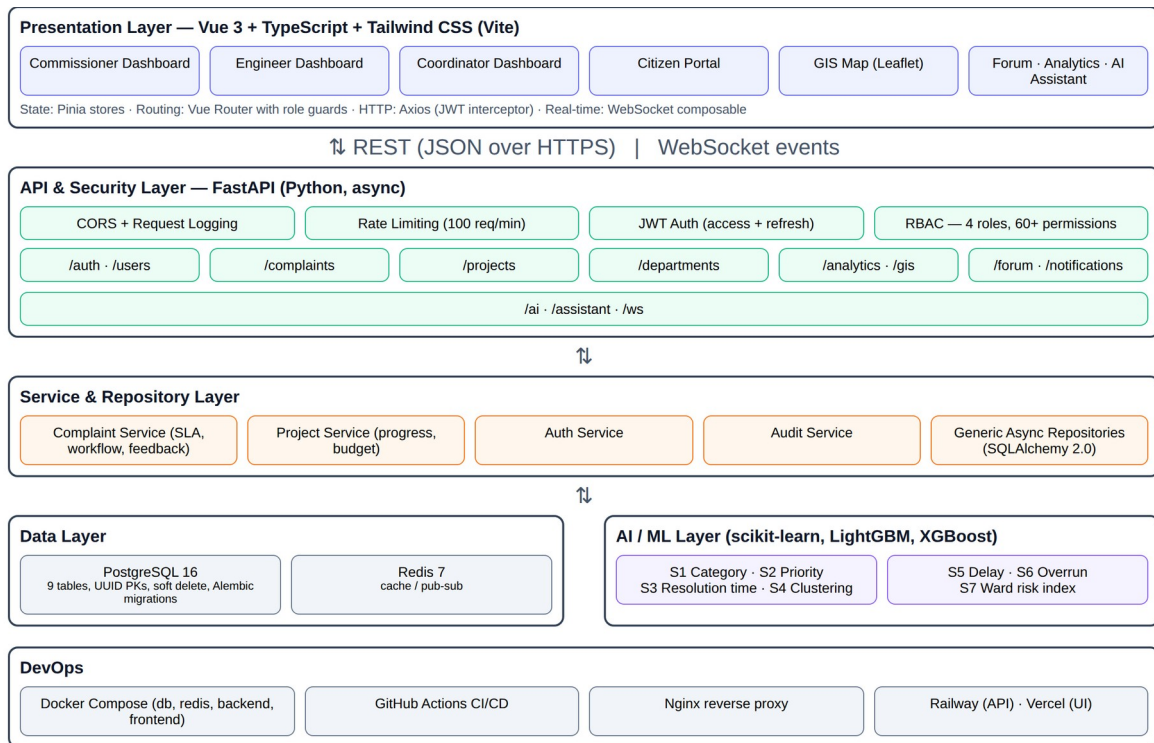


Figure 2.1: Layered System Architecture of the CivilInc Platform.

Table 2.2: Technology stack used in CivilInc

Layer	Technologies
Frontend	Vue 3 (Composition API), TypeScript, Vite, Tailwind CSS, Pinia (state management), Vue Router, Axios, Leaflet (maps), Lucide icons
Backend	Python, FastAPI, Uvicorn/Gunicorn, Pydantic v2, SQLAlchemy 2.0 (async), Alembic migrations, AsyncPG
Security	JWT (python-jose), bcrypt password hashing (passlib), role-based access control, rate limiting, CORS, structured logging (structlog)
Database & cache	PostgreSQL 16, Redis 7
AI / Data	pandas, NumPy, scikit-learn, LightGBM, XGBoost, joblib, NLTK; Anthropic Claude API (optional) for the assistant
DevOps	Docker, Docker Compose, Nginx, GitHub Actions, Railway (API) and Vercel (UI) deployment targets, Prometheus client for metrics
Testing	pytest, pytest-asyncio, httpx AsyncClient, factory-boy, pytest-cov

2.3 Database Design

The relational schema was designed to model the municipal domain while remaining flexible for future extension. All entities inherit from a common base model that provides a **UUID primary key**, creation and update **timestamps**, and **soft-delete** support so that records are never physically removed – an important requirement for auditability in government systems. Frequently filtered combinations of columns (for example status and priority, or ward and category) are covered by composite indexes. Schema changes are versioned using Alembic migrations.

Table 2.3: Core database tables

Table	Purpose and key fields
users	All actors of the system; email, phone, hashed password, role, department, ward, preferences, failed-login counter and lock-out time.
departments	Municipal departments (Roads, Water Supply & Sewerage, Electricity & Streetlights, Parks, Buildings & Planning, Solid Waste Management) with SLA hours, budget and performance counters.
complaints	Complaint content, category, status, priority, location (ward, zone, latitude/longitude), assignee, SLA due time and breach flag, escalation level, AI predictions and citizen feedback.
complaint_history	Every status change or comment on a complaint, with the actor and timestamp – the complaint’s timeline.
projects	Infrastructure works with phase, schedule, estimated/approved/actual cost, completion %, contractor, source of funds, GIS geometry and AI risk scores.
forum_threads, forum_comments	Cross-departmental and public discussion threads with pinning and moderation.
notifications	Per-user notifications with read/unread state.
audit_logs	Immutable record of sensitive actions for accountability.

2.4 Authentication and Role-Based Access Control

Security was treated as a first-class requirement. Users authenticate with email and password; passwords are stored only as **bcrypt** hashes. On successful login the API issues a short-lived **JWT access token** (30 minutes) carrying the user’s role and department, together with a longer-lived **refresh token** (7 days). The frontend attaches the access token to every request through an Axios interceptor and transparently refreshes it when it expires.

Authorisation is enforced through a **permission matrix** of 55 fine-grained permissions grouped into domains such as users, departments, projects, complaints, forum, analytics, AI, GIS, notifications, audit and budget. Each API endpoint declares the permission it needs using a FastAPI dependency, so the same rule is applied consistently regardless of how the endpoint is called. On the frontend, Vue Router guards hide pages a role cannot access.

Table 2.4: Summary of the role-permission matrix

Role	Scope of access
Commissioner	Full visibility: all users, departments, projects and complaints; city-wide analytics and export; approve budgets and projects; AI insights, override and retraining; broadcast notifications; full audit log.
Engineer	All complaints and projects; create and update own projects; assign, close and escalate complaints; department analytics; AI predictions and override; department audit log; update own budget.
Coordinator	View and triage all complaints; assign and close; update own projects; department analytics; view AI insights; all GIS layers.

Role	Scope of access
Citizen	Create, view and update only their own complaints; public projects and public GIS layer; forum participation; own notifications.

Additional safeguards include an in-process **rate limiter** (100 requests per minute per client, returning HTTP 429 when exceeded), **request logging middleware** that tags every request with an ID and response time, account lock-out after repeated failed logins, strict **CORS** configuration, and an **audit service** that records sensitive operations.

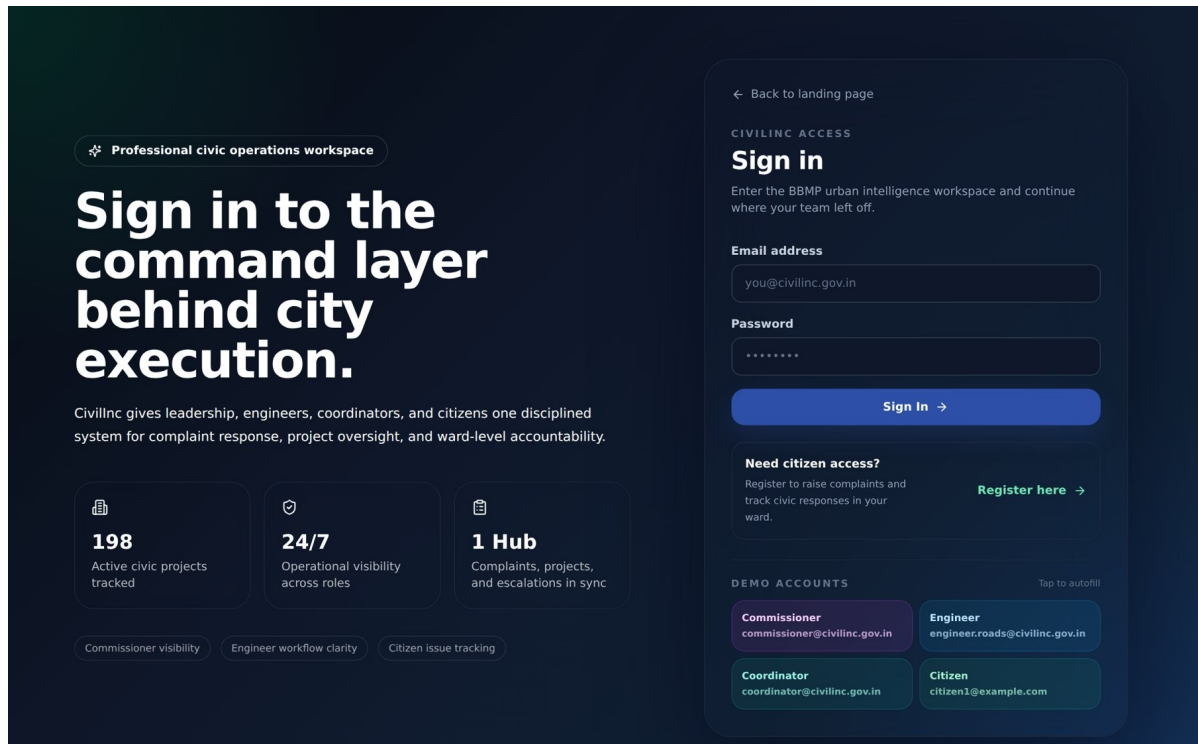


Figure 2.2: Secure Sign-in Screen with Role-based Demo Accounts.

2.5 Complaint Management Workflow

The heart of Civillnc is an end-to-end digital grievance redressal workflow. A complaint is modelled as a **state machine** with the states Pending, Under Review, Assigned, In Progress, Resolved, Closed, Rejected and Escalated. Each transition is permitted only to roles that hold the corresponding permission, and every transition is written to the complaint history and audit log and pushed to the relevant users as a notification.

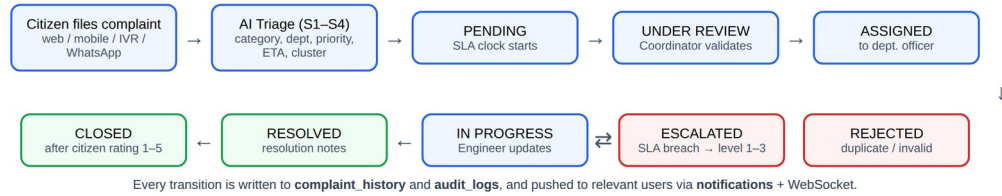


Figure 2.3: Complaint Lifecycle and State Transitions in CivillInc.

When a citizen submits a complaint, the backend validates the input, generates a unique complaint number, calculates the **SLA due time** and stores the record. The AI triage service (Section 2.11) predicts the category, responsible department, priority, expected resolution time and cluster, and these predictions are stored alongside the complaint so that a coordinator can accept or override them. If the SLA due time passes before resolution, the complaint is flagged as breached and can be escalated through up to three levels. Once resolved, the citizen can rate the resolution from 1 to 5 and leave feedback, which feeds into department performance metrics.

CivilInc

Bengaluru civic command center

LOGGED IN AS

Ravi Verma

Citizen

Dashboard

Complaints

Projects

GIS Map

Forum

Notifications

Settings

< Collapse

URBAN COMMAND PLATFORM

Bengaluru Municipal Corporation

+ Report Issue

R Ravi

Citizen

CITIZEN REPORTING

< Back

Report an Issue

Submit a civic complaint to BBMP with enough detail for faster triage and routing.

Title *

Briefly describe the issue (min 10 chars)

0/200

Description *

Provide detailed description including location, duration, impact...

0/2000

Category *

Select category...

Ward Number

e.g. Ward-42

Address / Location

Full address or landmark

Latitude (optional)

12.9716

Longitude (optional)

77.5946

Cancel

Submit Complaint

Figure 2.4: Citizen Complaint Registration Form with Location Details.

CivilInc

Bengaluru civic command center

LOGGED IN AS

Dr. Rajesh Kumar

Commissioner

Dashboard

Complaints

Projects

GIS Map

Analytics

Forum

Officers

Departments

Notifications

< Collapse

URBAN COMMAND PLATFORM

Bengaluru Municipal Corporation

D Dr.

Commissioner

COMPLAINT OPERATIONS

Complaints

4,005 total complaints across the platform.

Q Search complaints...

All Status

All Priority

All Categories

ID	TITLE	CATEGORY	PRIORITY	STATUS	WARD	FILED	
CMP-0000005	Park benches broken and garden not ...	Parks	low	resolved	Ward-55	4/10/2026	View
CMP-0000004	Blocked storm drain causing flooding i...	Drainage	high	under review	Ward-42	4/10/2026	View
CMP-0000003	Streetlight not working near Cubbon P...	Electricity	medium	pending	Ward-25	4/10/2026	View
CMP-0000002	Water supply disruption in Indiranagar ...	Water Supply	high	in progress	Ward-9	4/10/2026	View
CMP-0000001	Large pothole causing accidents near ...	Roads	critical	assigned	Ward-29	4/10/2026	View
CMP-0040313	Streetlight not working near Sankey Ta...	Electricity	medium	pending	Ward-139	31/12/2024	View
CMP-0038413	Power outage in Byatarayanapura — tr...	Electricity	medium	under review	Ward-100	31/12/2024	View
CMP-0007509	Road caving in near NIMHANS Circle d...	Roads	medium	pending	Ward-116	31/12/2024	View
CMP-0047656	Playground equipment rusted and brok...	Parks	low	under review	Ward-99	31/12/2024	View
CMP-0045166	Tree fallen blocking park entrance at B...	Parks	high	pending	Ward-141	31/12/2024	View

Figure 2.5: Complaint Register with Search and Filters by Status, Priority and Category.

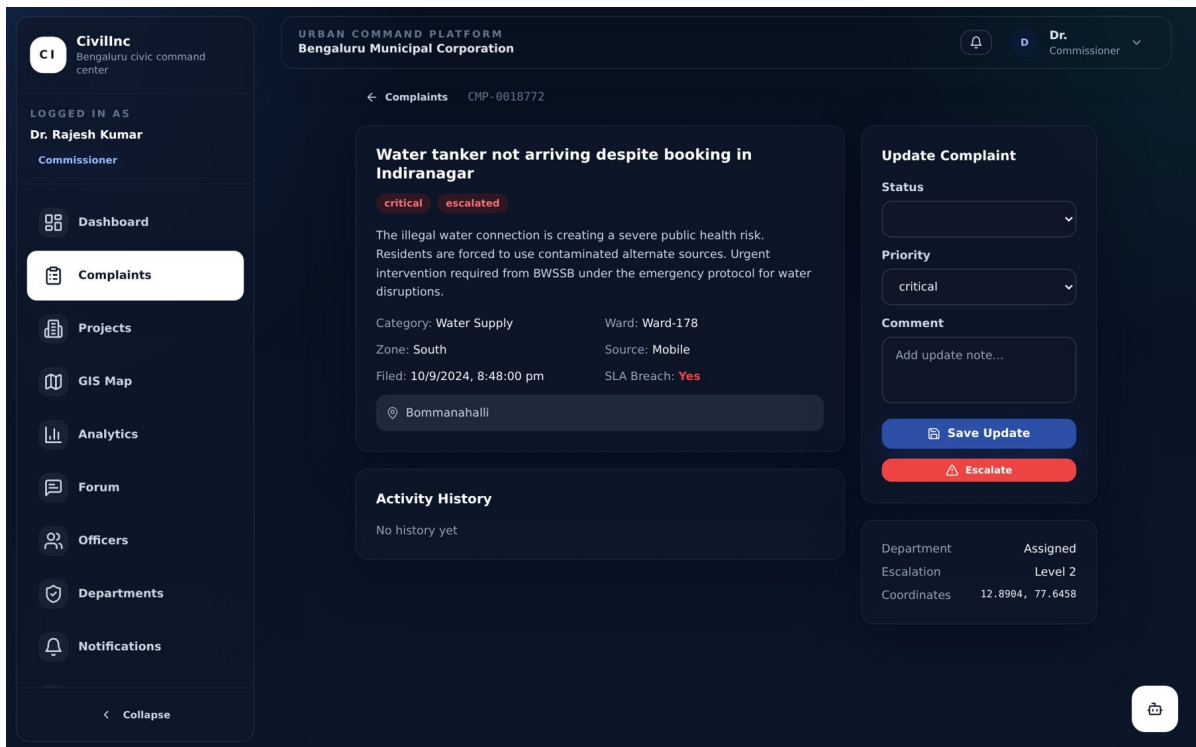


Figure 2.6: Complaint Detail View with Status Update, Escalation and SLA Information.

2.6 Role-Based Dashboards

Each of the four roles receives a dashboard tailored to its responsibilities, which is selected automatically after login by a dashboard router component.

The **Commissioner Dashboard** provides an executive, city-wide view: total complaints and pending count, resolution rate, critical issues and SLA breaches, active and at-risk projects, complaints by category, and the highest-risk wards identified by the AI ward risk index. This directly addresses the need for real-time oversight expressed by the administration.

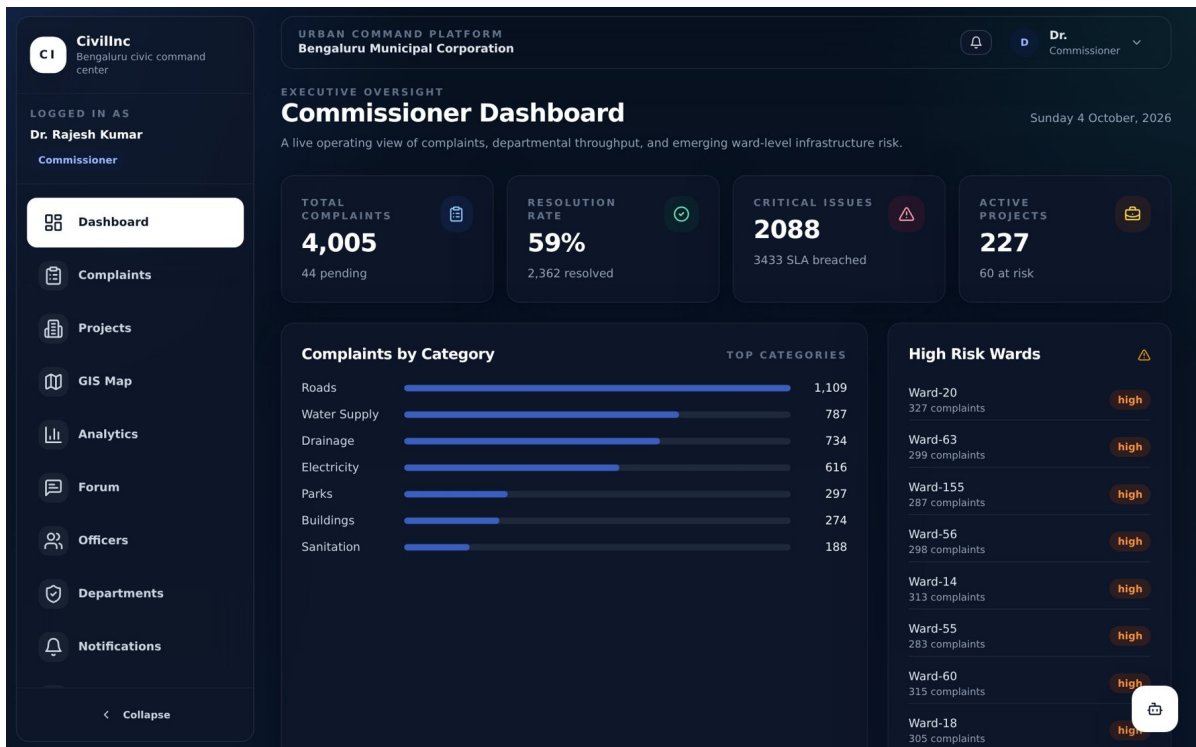


Figure 2.7: Commissioner Dashboard showing City-wide KPIs and High-Risk Wards.

The **Engineer Dashboard** focuses on execution: complaints assigned to the engineer's department, work in progress, resolved counts and a list of at-risk projects with their AI-predicted delay probability. The **Coordinator Dashboard** presents the triage queue – complaints pending review – so that coordinators can validate and route them quickly. The **Citizen Dashboard** offers quick actions to report an issue, track complaints, view the map and follow ward-level projects.

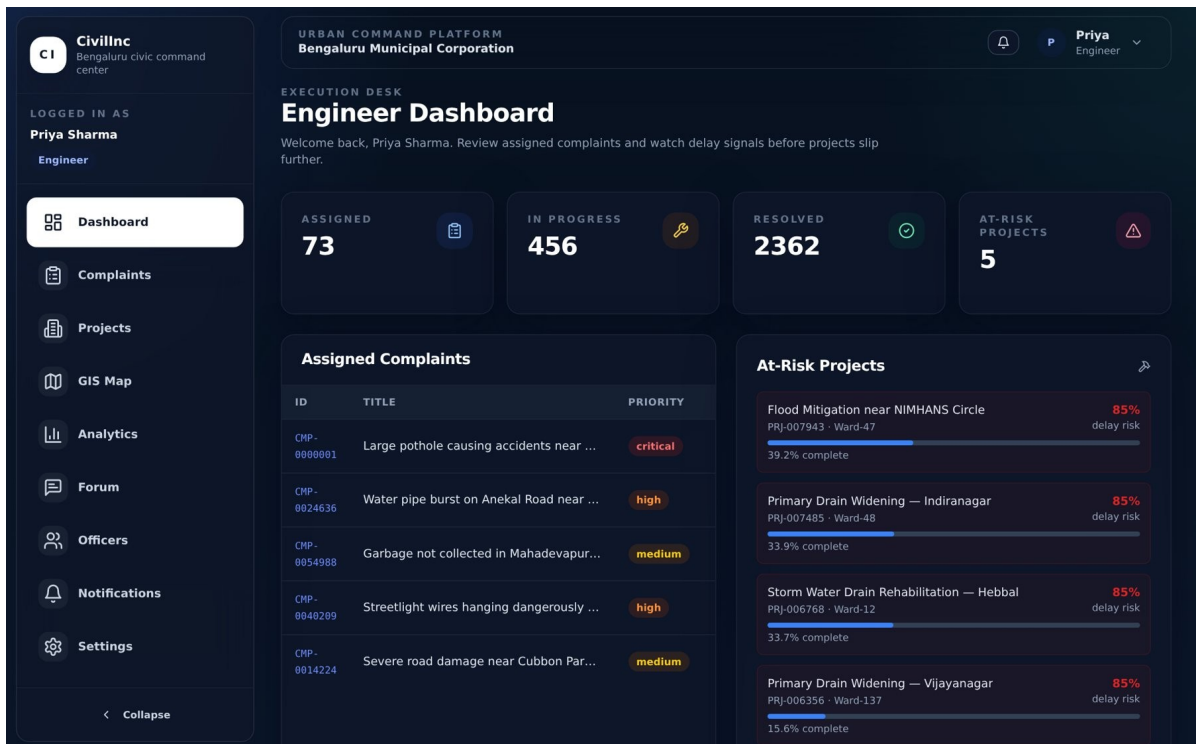


Figure 2.8: Engineer Dashboard with Assigned Complaints and At-Risk Projects.

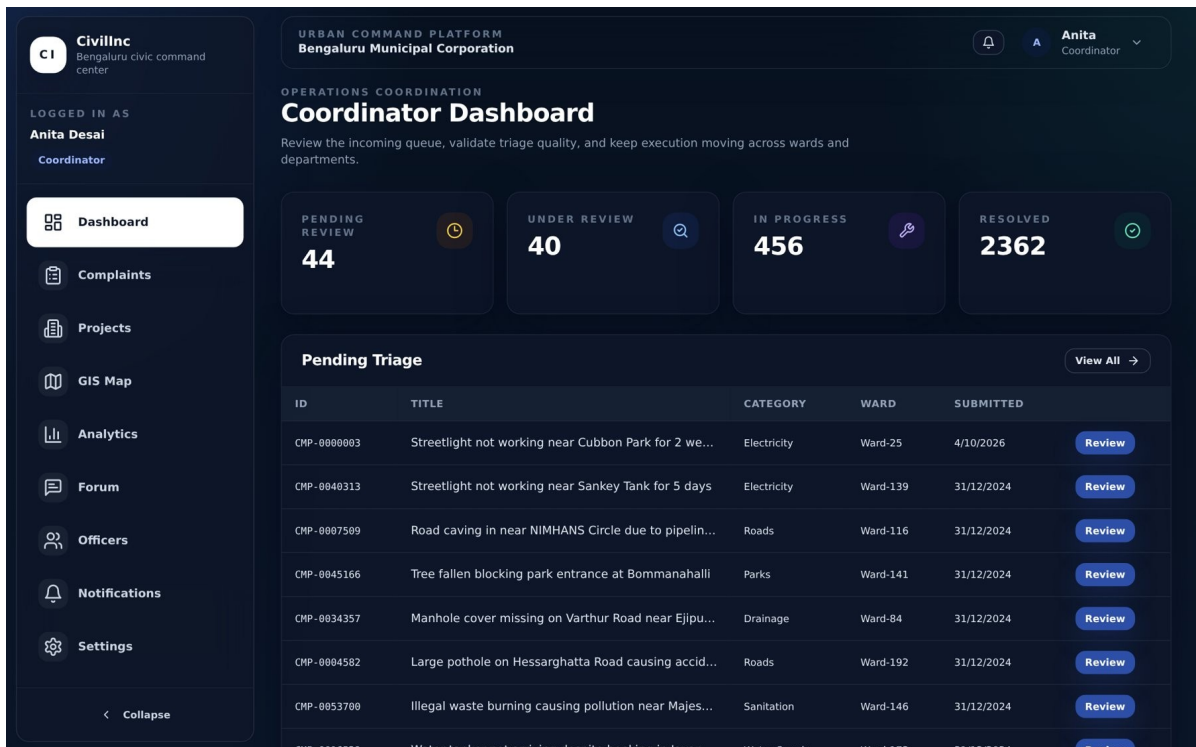


Figure 2.9: Coordinator Dashboard showing the Pending Triage Queue.

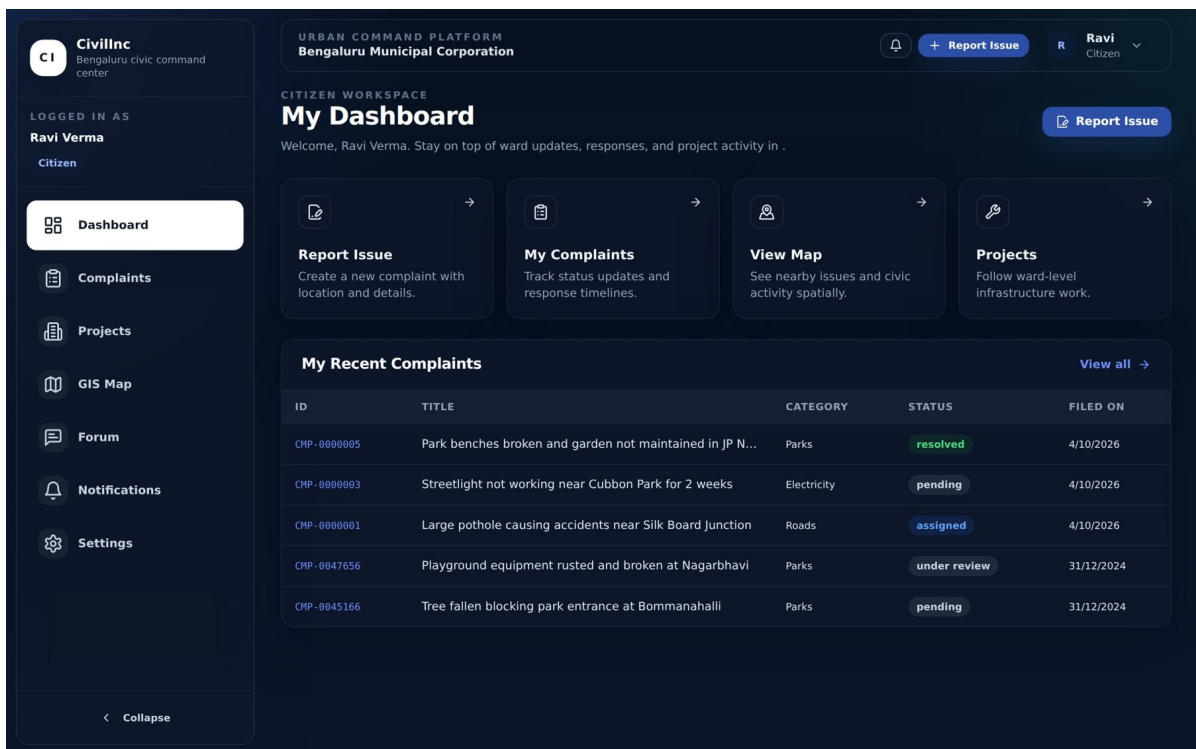


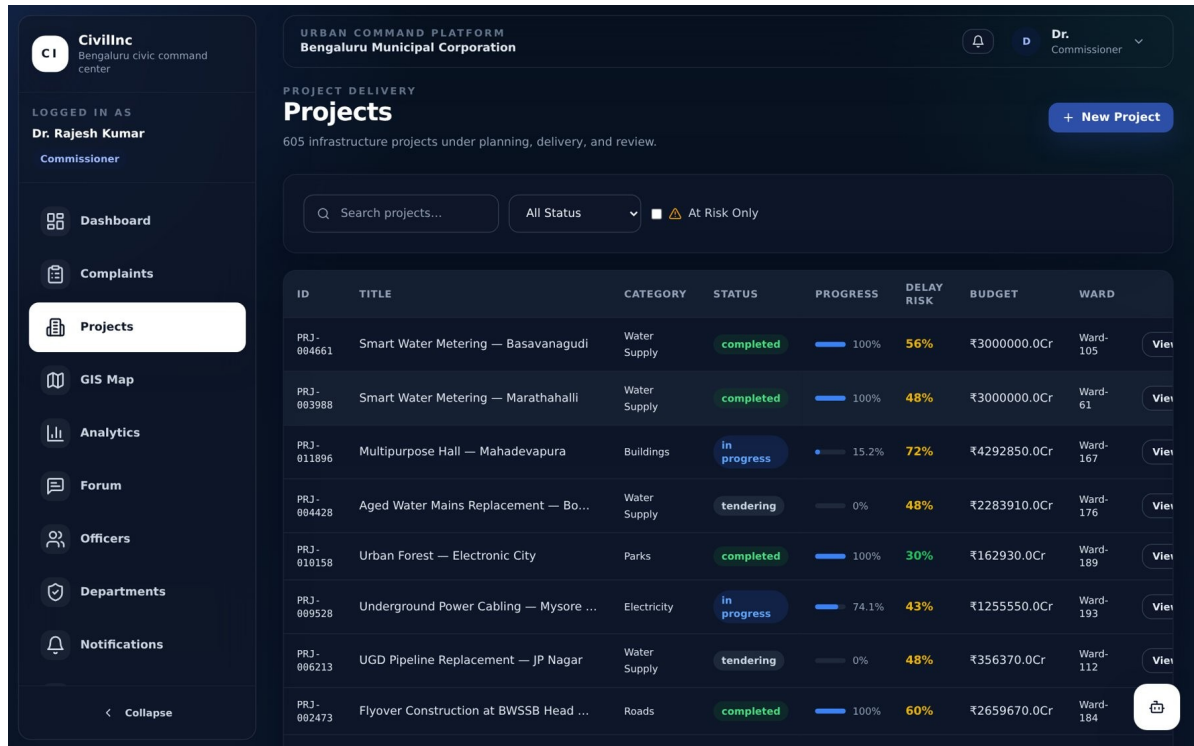
Figure 2.10: Citizen Dashboard with Quick Actions and Complaint Tracking.

2.7 Infrastructure Project and Budget Monitoring

Beyond grievances, Civillnc tracks infrastructure projects such as road widening, storm-water drain rehabilitation, water-pipeline replacement and LED streetlight installation. For each project the platform records the category, department, ward, schedule (planned and actual

dates), estimated, approved and actual cost, completion percentage, contractor and source of funds (for example Smart City, AMRUT 2.0 or the corporation’s own funds).

Engineers update progress and actual expenditure from the project detail page, and the system recalculates **budget utilisation**. Each project also displays an **AI Risk Assessment** panel showing the predicted delay probability, expected budget overrun and an overall risk score and category, enabling officials to intervene before a project slips. The project register can be filtered to show only at-risk projects.



ID	TITLE	CATEGORY	STATUS	PROGRESS	DELAY RISK	BUDGET	WARD
PRJ-004661	Smart Water Metering — Basavanagudi	Water Supply	completed	100%	56%	₹3000000.0Cr	Ward-105
PRJ-003988	Smart Water Metering — Marathahalli	Water Supply	completed	100%	48%	₹3000000.0Cr	Ward-61
PRJ-011896	Multipurpose Hall — Mahadevapura	Buildings	In progress	15.2%	72%	₹4292850.0Cr	Ward-167
PRJ-004428	Aged Water Mains Replacement — Bo...	Water Supply	tendering	0%	48%	₹2283910.0Cr	Ward-176
PRJ-010158	Urban Forest — Electronic City	Parks	completed	100%	30%	₹162930.0Cr	Ward-189
PRJ-009528	Underground Power Cabling — Mysore ...	Electricity	In progress	74.1%	43%	₹1255550.0Cr	Ward-193
PRJ-006213	UGD Pipeline Replacement — JP Nagar	Water Supply	tendering	0%	48%	₹356370.0Cr	Ward-112
PRJ-002473	Flyover Construction at BWSSB Head ...	Roads	completed	100%	60%	₹2659670.0Cr	Ward-184

Figure 2.11: Infrastructure Project Register with Progress, Delay Risk and Budget.

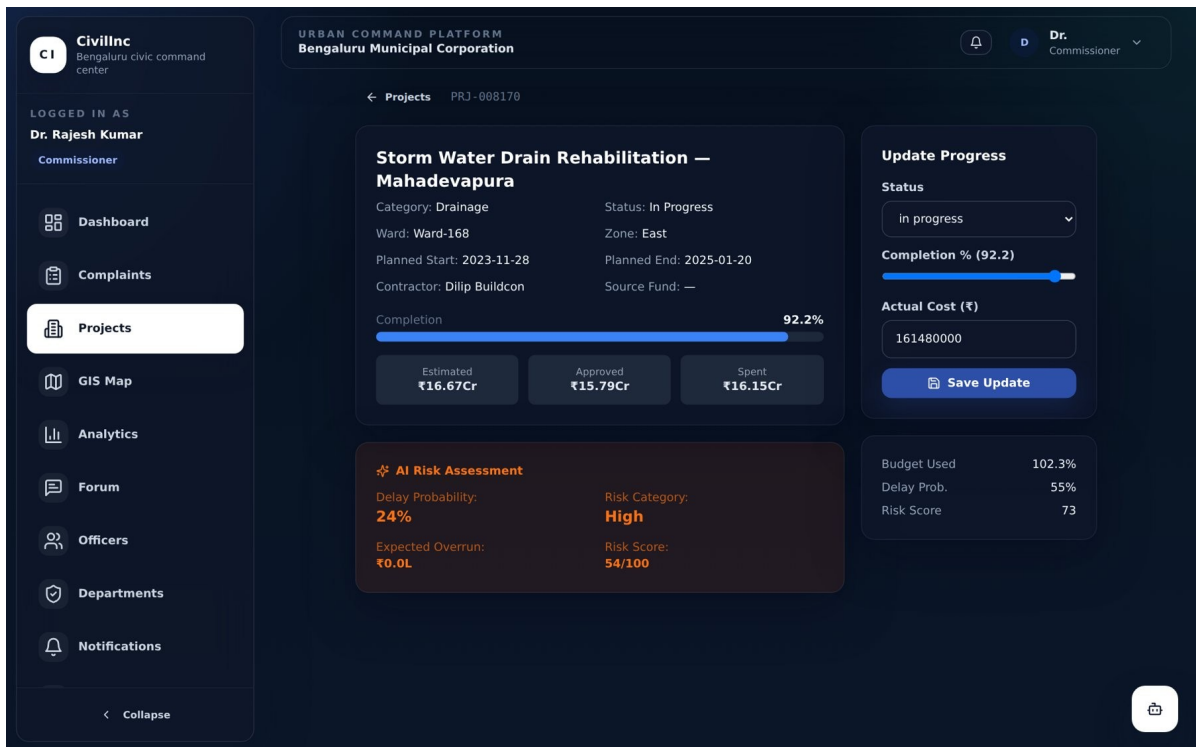


Figure 2.12: Project Detail with Progress Update and AI Risk Assessment.

2.8 GIS Mapping and Spatial Visualisation

Civic problems are inherently spatial. CivilInc includes a **GIS Map** module built with Leaflet and OpenStreetMap tiles. Dedicated GIS endpoints return complaint and project locations as point layers, and a heat-map endpoint aggregates complaint density. Markers are colour-coded by priority (critical, high, medium, low), and users can toggle between complaint and project layers. Commissioners, engineers and coordinators can view all layers, whereas citizens see only the public layer. The map makes clusters of problems – for example repeated drainage complaints along one road – immediately visible.

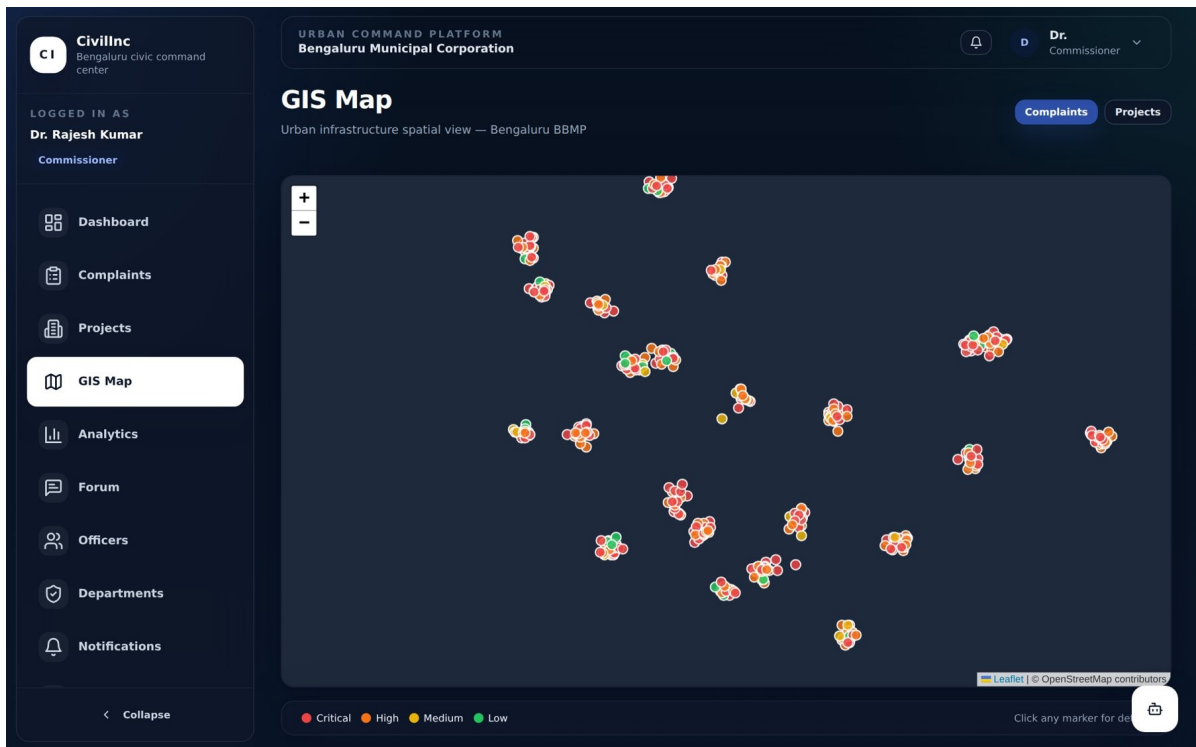


Figure 2.13: GIS Map showing Geo-tagged Complaints colour-coded by Priority.

2.9 Analytics and Cross-Departmental Coordination

The **Analytics** module exposes REST endpoints for a city-wide overview and for complaints by category, by status and by ward, projects by status and department performance. The analytics page brings these together with KPIs such as resolution rate, SLA breaches and budget utilisation, along with top complaint wards and AI-identified high-risk wards.

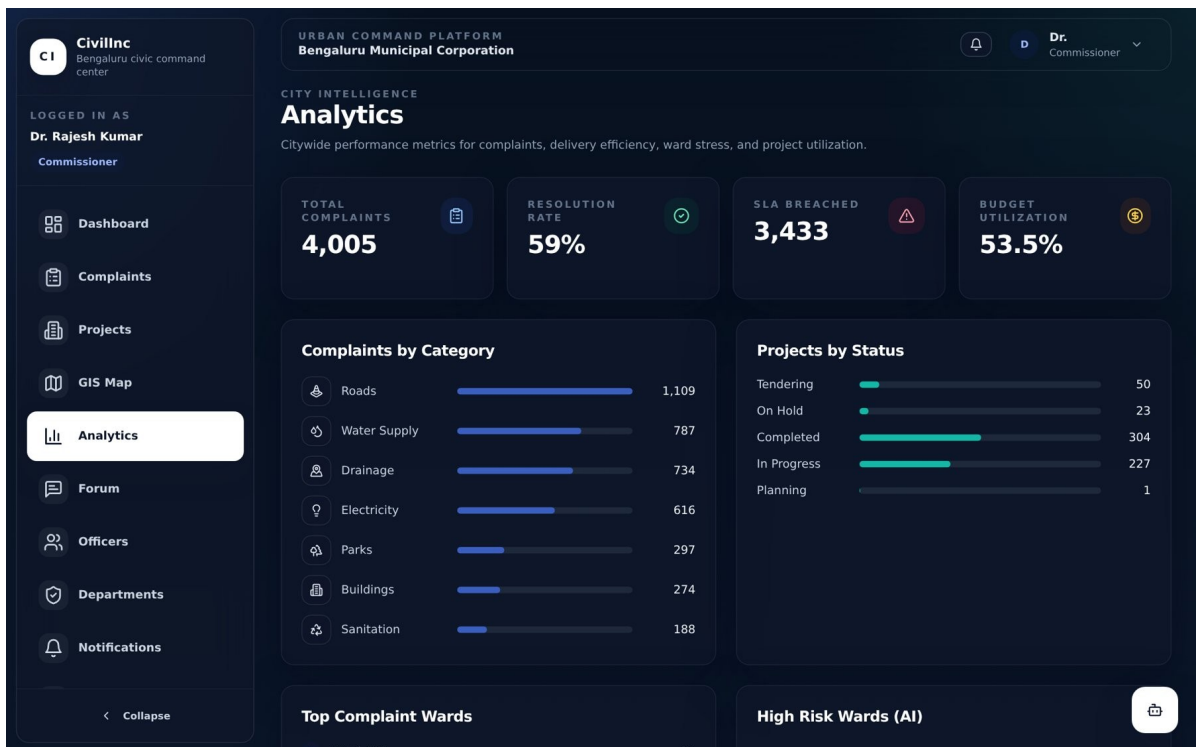


Figure 2.14: Analytics Module with Category, Project Status and Budget Metrics.

To support **cross-departmental coordination**, the platform provides a **Departments** view summarising each department’s complaints, projects, SLA hours and budget, and a **Discussion Forum** where officials can create threads – for example, a monsoon preparedness thread that coordinates drain clearing and road patching across departments. Commissioners can pin important threads and moderate discussions. A **notification centre** and a **WebSocket hub** deliver real-time updates when complaints are created, assigned or escalated; connections can join “rooms” per department or ward so that only relevant users receive each event.

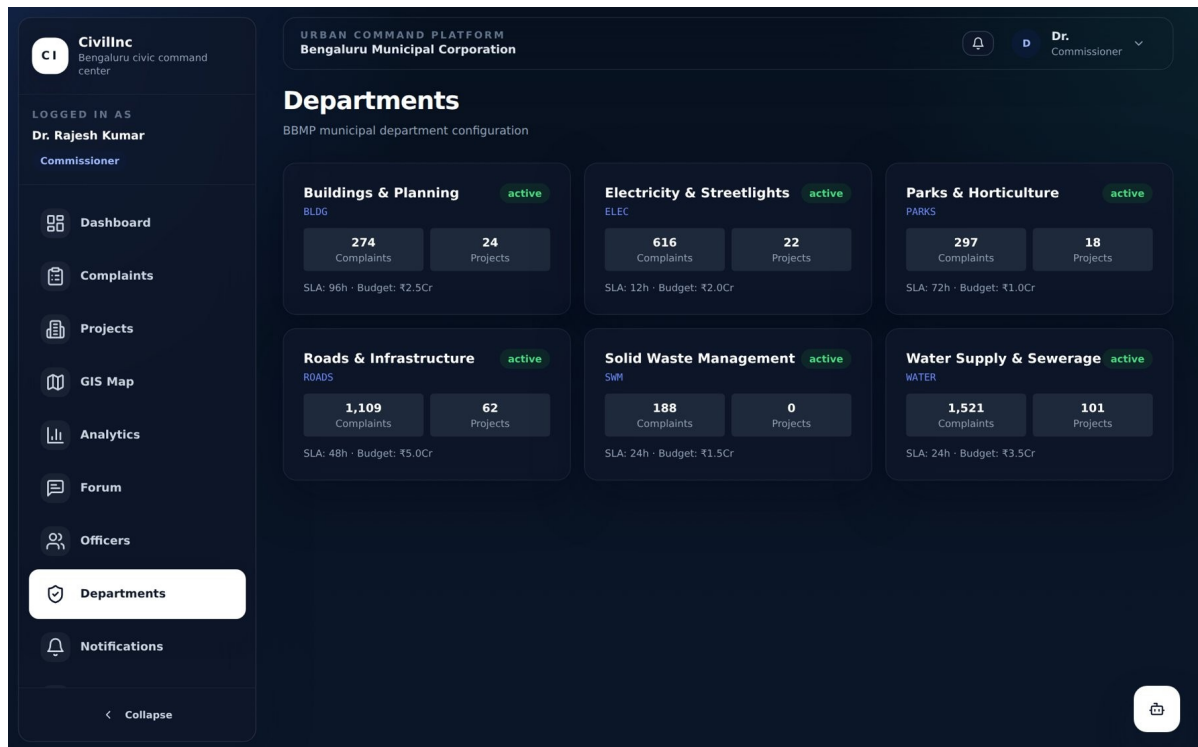


Figure 2.15: Department Overview with Complaints, Projects, SLA and Budget.

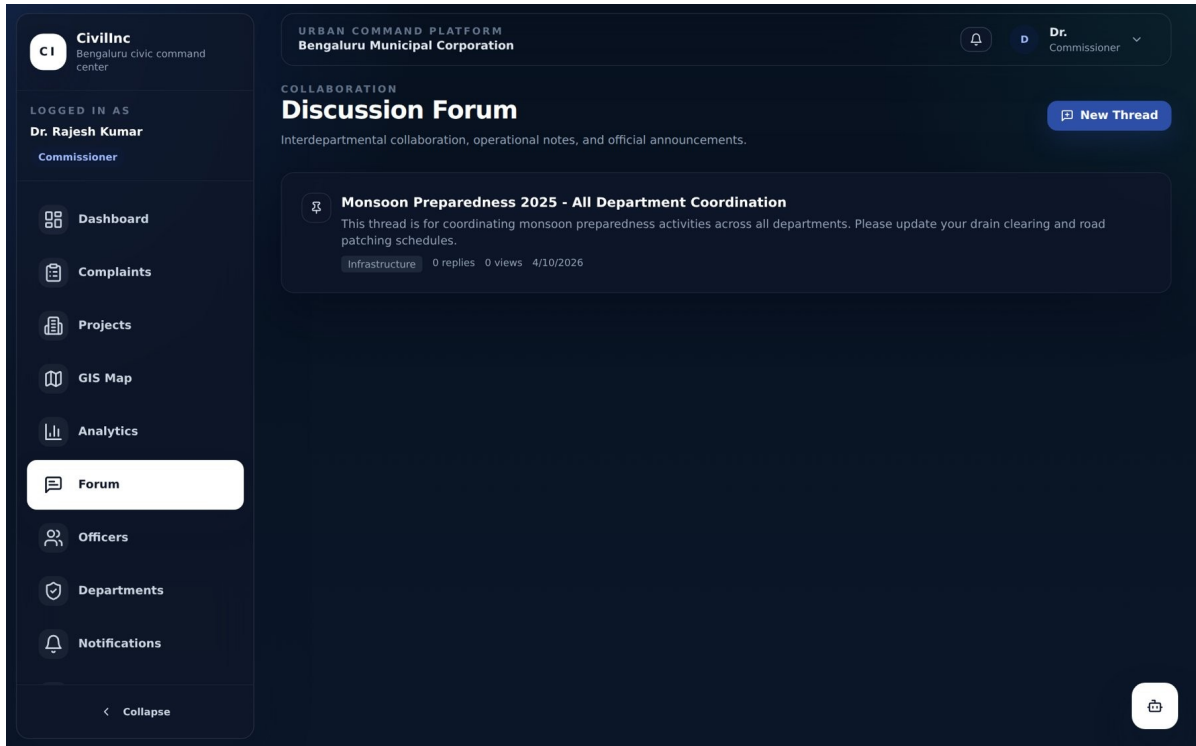


Figure 2.16: Inter-departmental Discussion Forum.

2.10 Data Engineering and Data Quality

Machine-learning models require substantial historical data. Because live municipal records could not be released for a student prototype, a reproducible **data engineering pipeline** was built to create a realistic dataset. Real geographic reference data – a 198-ward urban boundary dataset with OpenStreetMap-verified ward centroids, Indian city and state datasets – was combined with **statistical distributions published in public reports** (municipal annual and budget reports, Smart Cities Mission and NITI Aayog urban infrastructure reports, and open 311 service-request summaries) for complaint categories, channel mix, SLA targets, resolution times, project delay rates and budget utilisation. The 198-ward reference geography was used because ward-level open data is publicly available for it; the pipeline is parameterised so that Shivamogga’s own ward boundaries and records can be substituted directly.

Four Python ETL scripts orchestrated by a master pipeline generate five processed datasets. Every record carries **lineage columns** recording its geographic and statistical source, and the pipeline writes a **data provenance report** and a **data quality report** (row and column counts, missing values, distributions and date ranges). A data dictionary documents every field.

Table 2.5: Datasets produced by the ETL pipeline

Dataset	Records	Contents
complaints.csv	55,000	23 fields: text, category, sub-category, status, priority, ward, zone, coordinates, department, SLA, dates, resolution days, escalation, rating, source channel (2022–2024)
projects.csv	12,000	31 fields: costs, schedule, completion, delay days, contractor, fund source, monsoon-start flag, pace indicator

Dataset	Records	Contents
citizen_feedback.csv	30,000	Ratings, sentiment, feedback text, response and resolution times
officer_workload.csv	8,964	Monthly officer workload, resolution rate, SLA breaches, workload score
budgets.csv	36	Year-wise departmental allocation, utilisation and carry-forward (2019–2024)

The quality report on the complaints dataset confirmed no missing values in mandatory fields; resolution dates were missing only for open complaints (12,996 records) and ratings only where citizens did not give feedback (a feedback rate of 49.7%). The mean resolution time was 7.2 days (median 5.7 days), and the overall SLA breach rate was 85.1%, reflecting the strain on municipal services described in the source reports.

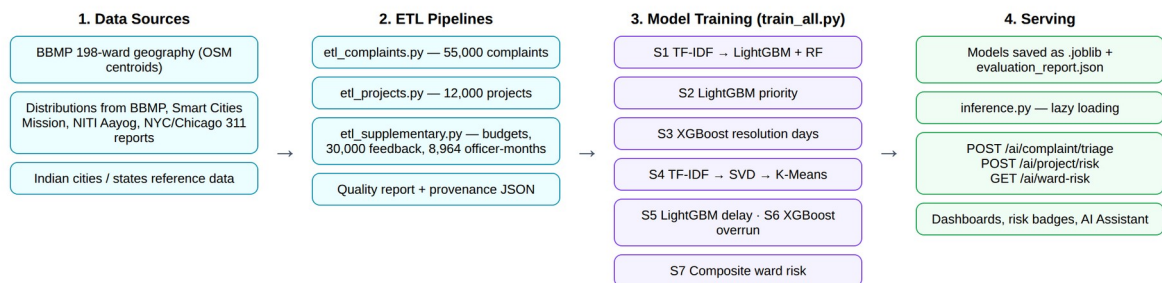


Figure 2.17: Data Engineering and Machine-Learning Pipeline of CivilInc.

2.11 AI and Machine-Learning Systems

Seven machine-learning systems were developed in a single training pipeline (`train_all.py`) and serialised with `joblib` for use by the backend inference service. Data was split 80:20 with stratification and fixed random seeds for reproducibility, and evaluation metrics are written to an evaluation report.

Table 2.6: AI systems implemented in CivilInc

#	System	Technique	Result on held-out data
S1	Complaint category classification	TF-IDF text features + LightGBM and Random Forest ensemble	Weighted F1 = 1.00 (target 0.85)

#	System	Technique	Result on held-out data
S2	Priority prediction	LightGBM on category, zone, channel, department, SLA and critical-keyword flags	Weighted F1 = 0.32 (target 0.80 – not met)
S3	Resolution time prediction	XGBoost regressor	MAE = 2.32 days, $R^2 = 0.66$ (target MAE < 3 days)
S4	Complaint clustering	TF-IDF → Truncated SVD → K-Means, k chosen by silhouette score	18 clusters, silhouette = 0.60, severity score per cluster
S5	Project delay prediction	LightGBM classifier on progress, pace, budget use, monsoon start, start delay	F1 = 0.75, ROC-AUC = 0.82
S6	Budget overrun prediction	XGBoost regressor	MAE = ₹7.11 lakh, $R^2 = 0.98$
S7	Ward risk scoring	Weighted composite index of normalised ward indicators	197 wards scored; 79 classified as high risk

Complaint triage (S1–S4). When a complaint is filed, the title is vectorised with TF-IDF and combined with encoded zone and channel features to predict the category; the category is then mapped to the responsible department and its SLA (for example 12 hours for electricity, 24 hours for water supply and solid waste, 48 hours for roads). The priority model adds a keyword feature that flags words such as “burst”, “collapse”, “exposed” and “accident”. The resolution-time regressor estimates the expected number of days, and the clustering model assigns the complaint to a thematic cluster. Each cluster profile stores its size, dominant categories and wards, SLA breach rate and a severity score, which helps identify systemic problems rather than isolated complaints.

Project risk (S5–S6). For each project the delay classifier outputs a probability of delay and the overrun regressor estimates the expected cost overrun. These are combined into a risk score (60% delay probability, 40% budget utilisation) and mapped to the categories low, medium, high and critical, which are shown on project pages and dashboards.

Ward risk index (S7). For every ward, seven indicators are min-max normalised and combined: complaint volume (25%), SLA breach rate (20%), average resolution time (15%), critical complaints (15%), escalations (10%), average project delay probability (10%) and budget utilisation (5%). Scores of 0.75 and above are classified as critical, 0.50 and above as high and 0.25 and above as medium. This index drives the “High Risk Wards” panel on the Commissioner dashboard.

```
Summary of ai/evaluation/evaluation_report.json

=====
CivilInc AI Training Pipeline – Evaluation Summary
Generated: 2026-06-20 14:44:54
=====
System 1 Category classification F1(weighted) = 1.0000 target 0.85 PASS
System 2 Priority prediction F1(weighted) = 0.3175 target 0.8 BELOW TARGET
System 3 Resolution time MAE = 2.315 days R² = 0.661 target MAE < 3 PASS
System 4 Complaint clustering k = 18 silhouette = 0.6
System 5 Project delay F1 = 0.7482 ROC-AUC = 0.8207
System 6 Budget overrun MAE = ₹7.11 lakh R² = 0.984
System 7 Ward risk scoring wards = 197 critical = 0 high = 79
=====
Models saved to ai/models/*.joblib · report: ai/evaluation/evaluation_report.json
```

Figure 2.18: Summary of the Model Evaluation Report produced by the Training Pipeline.

The models are exposed through three endpoints: POST /ai/complaint/triage, POST /ai/project/risk and GET /ai/ward-risk. The inference service loads all models lazily on first use and handles unseen categorical values gracefully. Figure 2.19 shows a live call to the triage endpoint for a new complaint about a burst water pipe; the system correctly identifies the category as water supply, routes it to the Water department with a 24-hour SLA and assigns high priority.

```
bash – AI triage endpoint

$ curl -X POST http://localhost:8000/api/v1/ai/complaint/triage \
  -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" \
  -d '{"title": "Water pipe burst flooding road near Gandhi Bazaar",
    "description": "Main water pipeline has burst and water is overflowing onto
    the road since morning, causing traffic issues and water wastage.",
    "zone": "Central", "source": "mobile"}'

{
  "category": "water_supply",
  "department": "WATER",
  "priority": "high",
  "resolution days": 1,
  "cluster_id": 0,
  "confidence": 1,
  "sla_hours": 24
}
```

Figure 2.19: Live AI Triage of a New Complaint through the REST API.

An important observation from the evaluation was that the perfect score of the category classifier reflects the relatively templated complaint text in the engineered dataset, and the priority model did not reach its target because priority in the source distributions depends on factors not present at the time of filing. These findings are discussed in Chapter 3 and are the main reasons why retraining on real municipal records is recommended before production use. The design therefore stores AI predictions separately and allows coordinators and engineers to **override** them.

2.12 Commissioner AI Assistant

To give senior officials quick answers without navigating multiple screens, a **Commissioner AI Assistant** was built into the interface. When a question is asked – for example “Which department has the highest backlog?” – the backend first builds a live snapshot of operational metrics from the database (complaint totals, pending and critical counts, SLA breaches, projects at risk and department-wise resolution rates). This snapshot is passed as context to a large language model (Anthropic Claude) with instructions to answer concisely using the actual figures. If the external model is unavailable, a **rule-based fallback** answers common questions about backlogs, project risk and department performance directly from the snapshot, ensuring the feature always works. The assistant is restricted to roles holding the AI insights permission.

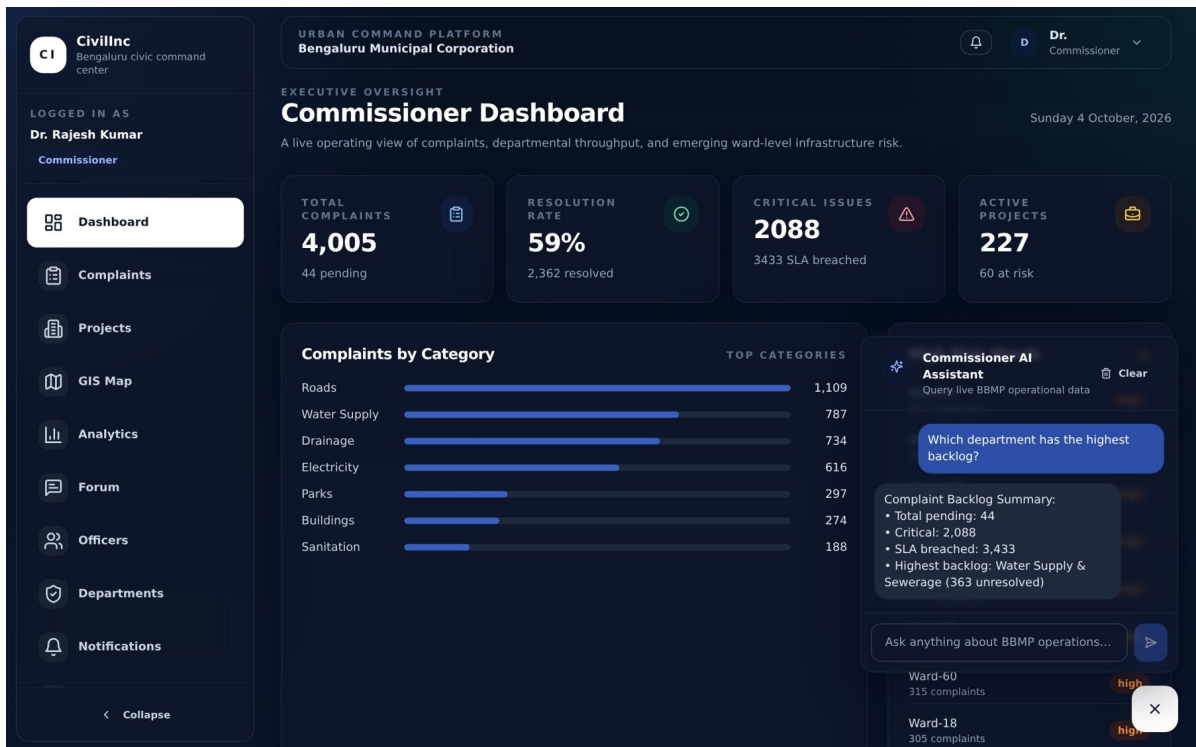


Figure 2.20: Commissioner AI Assistant answering a Backlog Query from Live Data.

2.13 Monitoring, DevOps and Deployment

The application exposes /health endpoints and a monitoring router with Prometheus-compatible metrics, and all requests are logged in structured form with a request ID and response time. Interactive API documentation is generated automatically by FastAPI (OpenAPI/Swagger) at /api/docs, covering 55 REST endpoints and the WebSocket endpoint.

For deployment, a **Docker Compose** file defines four services – PostgreSQL, Redis, backend and frontend – with health checks and persistent volumes, so that the complete platform can be started with a single command. A **GitHub Actions** pipeline runs backend unit and integration tests against a PostgreSQL service container with coverage reporting, builds the frontend, builds and pushes the backend Docker image, and deploys the API to Railway and the frontend to Vercel from the main branch. An Nginx production configuration is provided for on-premise hosting, which is often preferred by government organisations.

Automated tests were written with pytest and httpx for password hashing and token handling, the permission matrix for all four roles, authentication flows, and complaint rules such as “citizens can only see their own complaints” and “feedback is allowed only on resolved complaints”. The test configuration uses an isolated in-memory database with per-test transactions.

CHAPTER 3

LEARNINGS & IMPACT OF THE INDUSTRIAL TRAINING

The industrial training gave me a rare opportunity to see how a city government actually works and to build software for it. Understanding the roles of the Commissioner, departmental engineers and coordinators, and the expectations of citizens, taught me that the hardest part of a civic-tech system is not the code but modelling the real workflow correctly: who may do what, when a complaint should be escalated, and what information a decision-maker needs at a glance.

On the technical side, building CivilInc end-to-end strengthened my skills across the full stack. Designing a normalised relational schema with soft deletes, audit trails and indexes; writing an asynchronous FastAPI backend with a clean separation between routers, services and repositories; and building a reactive Vue 3 and TypeScript frontend with Pinia stores and route guards gave me practical experience of how production systems are structured. Implementing JWT authentication, a 55-permission RBAC matrix, rate limiting and audit logging taught me to treat security and accountability as core requirements, especially for systems that handle citizens' data.

The data engineering and machine-learning work taught me equally valuable lessons. I learned to design ETL pipelines that record lineage and produce data quality and provenance reports, and to train, evaluate and serve models for classification, regression and clustering. Most importantly, I learned to read evaluation results critically: a perfect F1 score was a warning sign of overly regular training data rather than a success, and a priority model that missed its target showed that some labels depend on information unavailable at prediction time. This experience reinforced the importance of honest evaluation, human override of AI outputs, and retraining on real data before deployment.

Working with containerisation, CI/CD pipelines, real-time WebSockets, GIS mapping and LLM integration with a reliable rule-based fallback exposed me to modern engineering practices that I will carry forward into my career.

In terms of impact, the prototype was reviewed by the Commissioner, Shivamogga City Corporation and Managing Director, Shivamogga Smart City Limited, who recognised four value propositions of CivilInc: an **end-to-end digital grievance redressal system** for citizens and officials, **cross-departmental coordination** through a centralised platform, **real-time analytics and dashboards** for performance tracking and decision-making, and **transparency and accountability** aligned with Smart City goals and governance reforms. The letter of recommendation describes the work as “commendable and promising for practical pilot implementation and future development” – the most meaningful outcome of this training for me.

Beyond technical knowledge, the training improved my ability to communicate with non-technical stakeholders, convert informal requirements into precise specifications, work independently on a large codebase of more than 9,000 lines across the frontend, backend and AI pipelines, and document design decisions clearly.

Chapter 4

CONCLUDING REMARKS

The industrial training undertaken with Shivamogga Smart City Limited bridged the gap between academic computer science and the practical challenges of urban governance. It allowed me to study a real public-service process, identify its shortcomings and independently design and build a working solution.

CivilInc brings complaint registration, AI-assisted triage, SLA tracking and escalation, project and budget monitoring, GIS mapping, analytics, a discussion forum, real-time notifications and a natural-language assistant together in a single secure, role-based platform. It demonstrates how digital tools can make municipal administration faster, more coordinated, more transparent and more accountable to citizens.

As a prototype, the platform has some limitations which also define the path for future development:

- Retraining the AI models on real, anonymised grievance and project records of Shivamogga City Corporation, and configuring the platform with Shivamogga's ward boundaries and departmental SLAs.
- Integration with existing government systems and channels such as helplines, WhatsApp and SMS gateways, and the city's command-and-control centre.
- Support for Kannada and voice input to make complaint registration accessible to all citizens, and photo-based issue detection.
- A Redis-backed distributed rate limiter and caching layer, and load testing for city-scale traffic, before production deployment.
- A controlled pilot in selected wards to measure improvements in resolution time and citizen satisfaction.

Overall, this industrial training was extremely valuable. It deepened my understanding of full-stack development, data engineering, machine learning and secure system design, and showed me how technology can be applied responsibly for public good. The recognition received from the Commissioner and Managing Director has strengthened my confidence and motivation to continue working in the field of civic technology, and the knowledge gained will benefit my academic and professional pursuits in the future.

REFERENCES

- Ministry of Housing and Urban Affairs, Government of India. (n.d.). *Smart Cities Mission*. <https://smartcities.gov.in/>
- FastAPI. (n.d.). *FastAPI documentation*. <https://fastapi.tiangolo.com/>
- Vue.js. (n.d.). *Vue.js guide*. <https://vuejs.org/guide/introduction.html>
- SQLAlchemy. (n.d.). *SQLAlchemy 2.0 documentation*. <https://docs.sqlalchemy.org/en/20/>
- PostgreSQL Global Development Group. (n.d.). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
- Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org/stable/>
- Ke, G., et al. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. *Advances in Neural Information Processing Systems* 30. <https://lightgbm.readthedocs.io/>
- Chen, T., & Guestrin, C. (2016). *XGBoost: A scalable tree boosting system*. *Proceedings of the 22nd ACM SIGKDD Conference*. <https://xgboost.readthedocs.io/>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. <https://datatracker.ietf.org/doc/html/rfc7519>
- Leaflet. (n.d.). *Leaflet documentation*. <https://leafletjs.com/reference.html>
- OpenStreetMap contributors. (n.d.). *OpenStreetMap*. <https://www.openstreetmap.org/>
- Docker Inc. (n.d.). *Docker Compose documentation*. <https://docs.docker.com/compose/>
- Anthropic. (n.d.). *Claude API documentation*. <https://docs.anthropic.com/>

PLAGIARISM / SIMILARITY REPORT

Turnitin Similarity Report

Insert the Turnitin similarity report screenshot here after running the final report through Turnitin.

Turnitin AI Writing Detection Report

Insert the Turnitin AI writing detection report screenshot here.

ANNEXURES

Appendix I: Additional Implementation Evidence

This annexure includes additional evidence of the implementation of the CivilInc platform, including further screens of the application and a summary of the REST API.

Appendix I(a): Public Landing Page of CivilInc

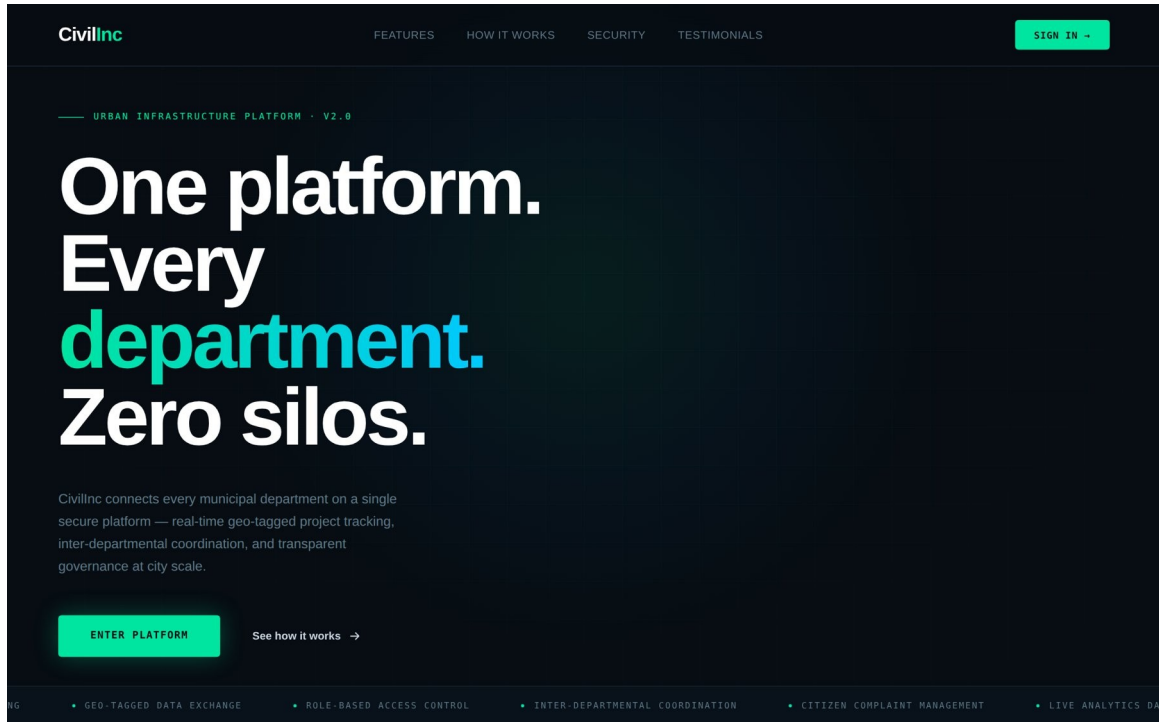


Figure A.1: Public Landing Page.

Appendix I(b): Officer Directory and Notification Centre

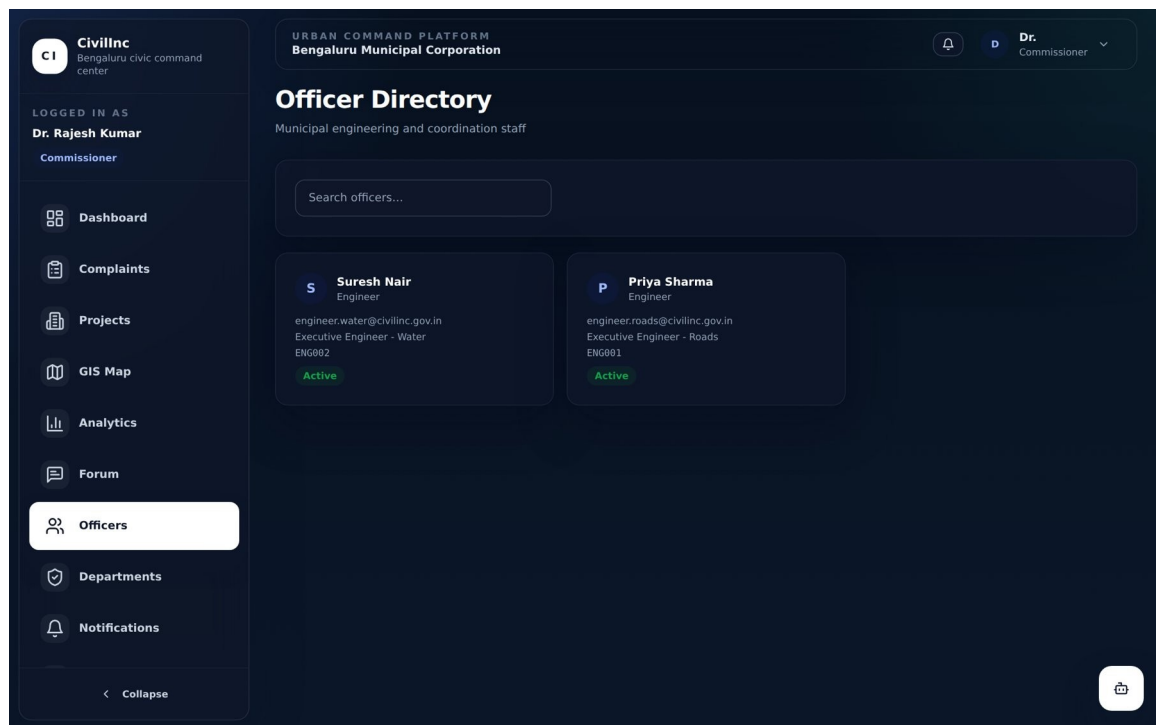


Figure A.2: Officer Directory.

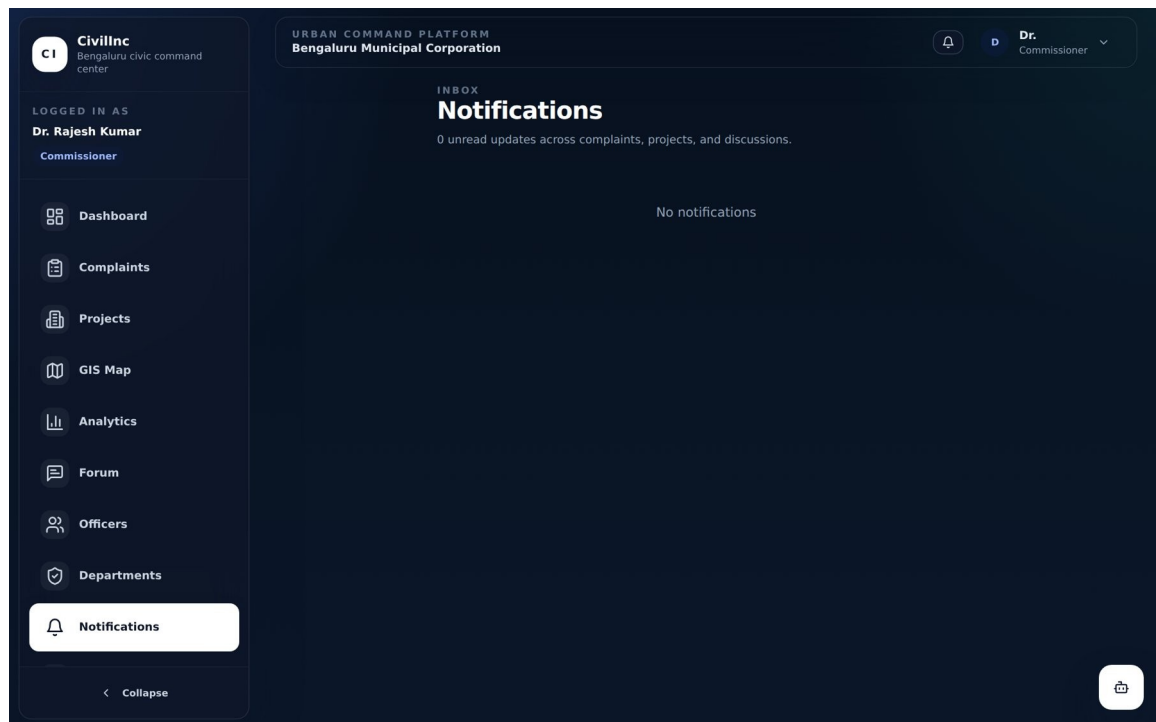


Figure A.3: Notification Centre.

Appendix I(c): Summary of REST API Endpoints

Router	Endpoints	Operations
/api/v1/auth	6	login, register, refresh token, current user, change password, logout

Router	Endpoints	Operations
/api/v1/users	5	list, create, get, update, soft-delete users
/api/v1/departments	5	list, create, get, update, department statistics
/api/v1/complaints	7	list (filters, pagination), create, get, update, feedback, history, escalate
/api/v1/projects	6	list, create, at-risk list, get, update, delete
/api/v1/forum	6	list/create threads, get thread, add/list comments, pin thread
/api/v1/notifications	4	list, unread count, mark read, mark all read
/api/v1/analytics	6	overview, complaints by category/status/ward, projects by status, department performance
/api/v1/gis	3	complaint points, project points, complaint heat-map
/api/v1/ai	4	complaint triage, project risk, ward risk, model health
/api/v1/assistant	2	natural-language query, weekly commissioner report
/api/v1/ws	1	WebSocket hub for real-time events